

OWASP Top 10

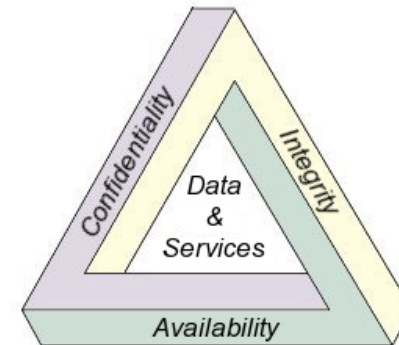
Introduction to web security

Objective

- Provide you with a quick introduction to web application security
- Increase your awareness and knowledge of security in general
- Show you that “any tester” can (and should) do security testing and not just leave it to the pen testers

What is Security?

- The traditional security areas of concern are:
 - Confidentiality
 - Integrity
 - Availability
 - Accountability.
- Security is dependent on context – different organisations have different needs.

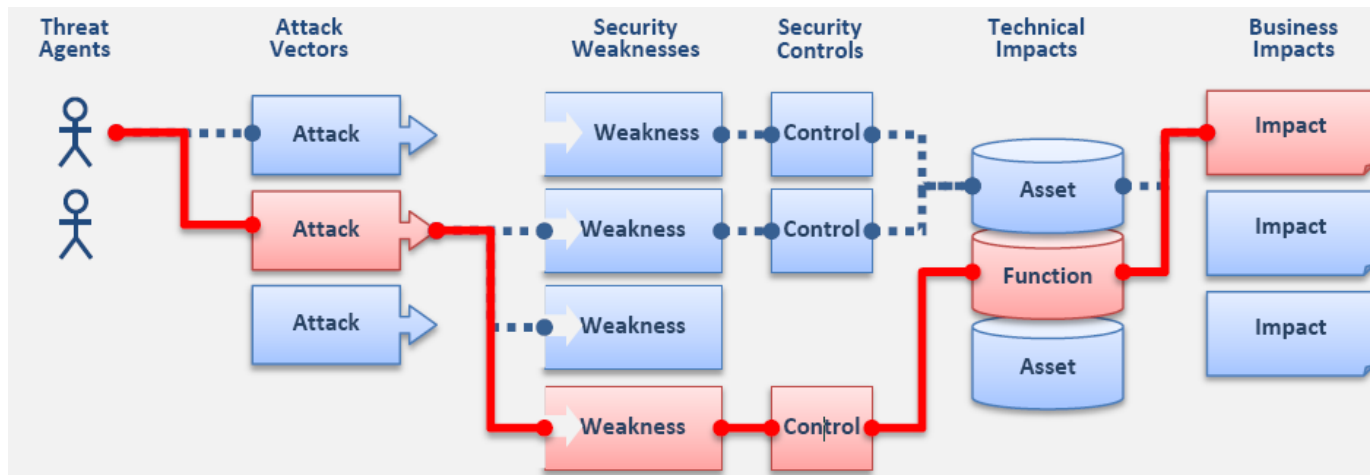


Types of Security

- Physical – doors, walls, locks etc.
- Network– OS, network, firewalls etc.
- Application – software at the application layer
- Perceived vs. Actual security
 - Security theater
 - FUD –Fear Uncertainty & Doubt
 - Absolutism – nothing is 'totally secure'

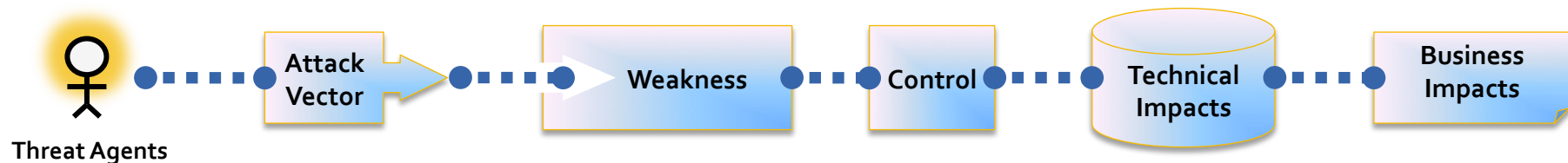
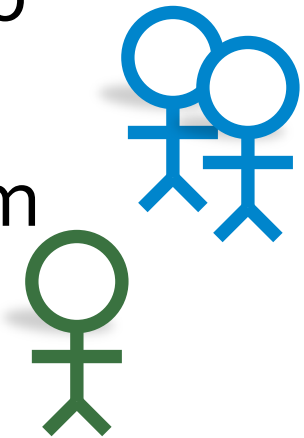
What Are Application Security Risks?

- Attackers can potentially use many different paths through your application to do harm to your business or organization. Each of these paths represents a risk that may, or may not, be serious enough to warrant attention.



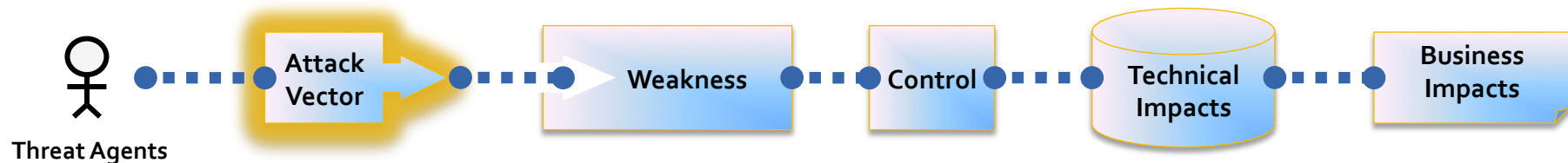
What is a Threat Agent?

- The term *Threat Agent* is used to indicate an individual or group that can manifest a threat. It is fundamental to identify who would want to exploit the assets of a company, and how they might use them against the company.



What is an Attack?

- Attacks are the techniques that attackers use to exploit the vulnerabilities in applications.



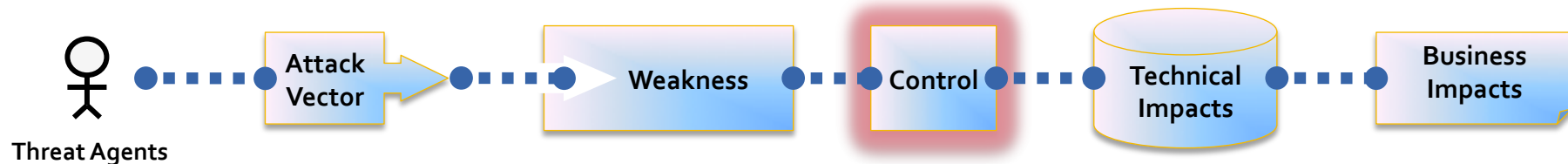
What is a Vulnerability?

- A vulnerability is a hole or a weakness in the application, which can be a design flaw or an implementation bug, that allows an attacker to cause harm to the stakeholders of an application. The term "vulnerability" is often used very loosely.



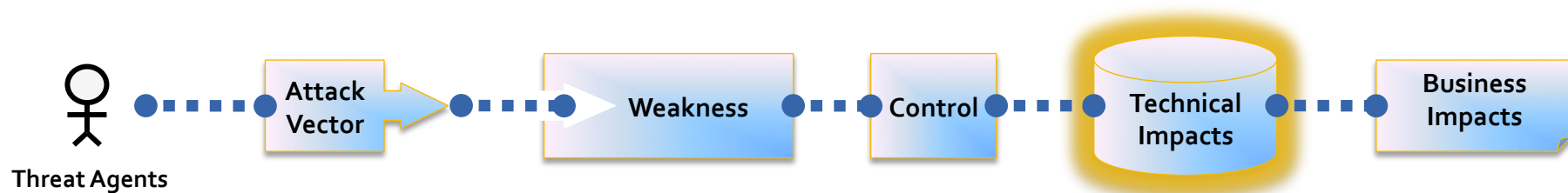
What is a Control?

- Controls are defensive technologies or modules that are used to detect, deter, or deny attacks.
- A weakness or design flaw of a control, or the lack of a necessary controls results in a vulnerability that can make the application susceptible to attacks.



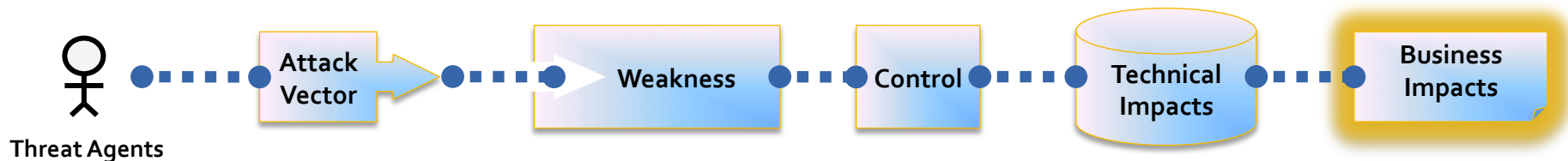
What is Technical impact?

- A technical impact is the system damage that results from a successful security breach. This is just the effect on the technology, not the business.

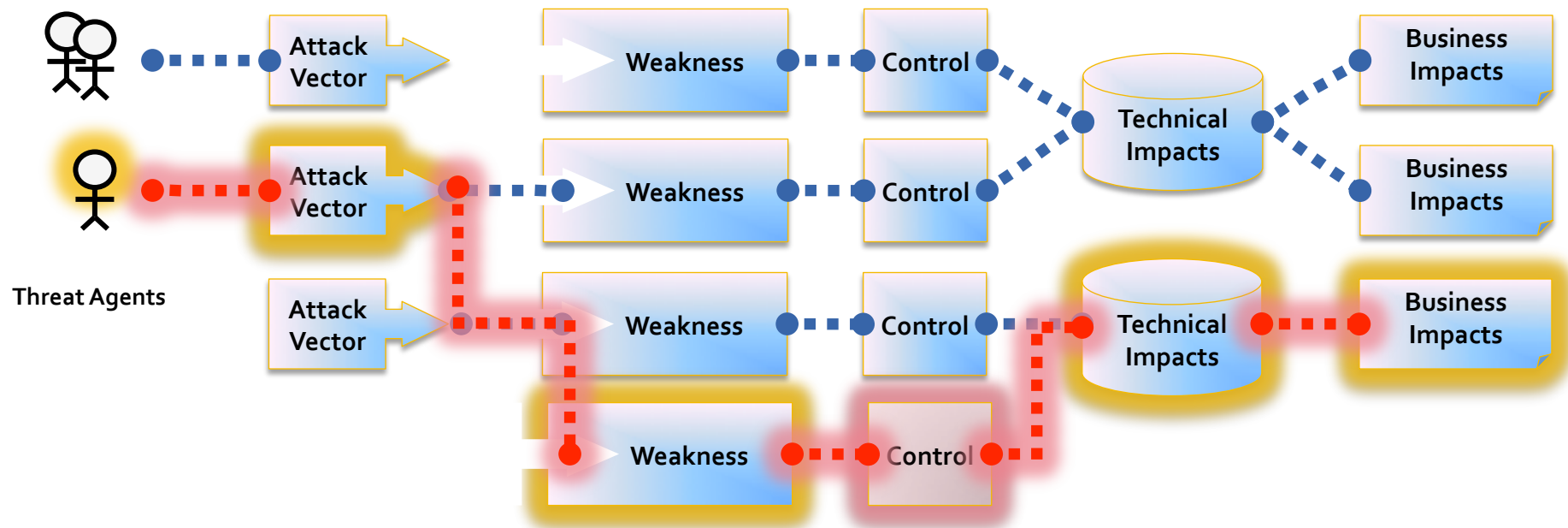


What is Business Impact?

- A business impact is the impact to a business that results from a successful attack.
- Generally this is in terms of money, lives, reputation, customers, or speed.
- The business risk is what justifies investment in fixing security problems.



Example of Security Risk



A *threat agent* detects a *weakness* in the application and its *controls*. The agent launches an *attack* that causes both *technical* and *business impacts*.

OWASP Top 10



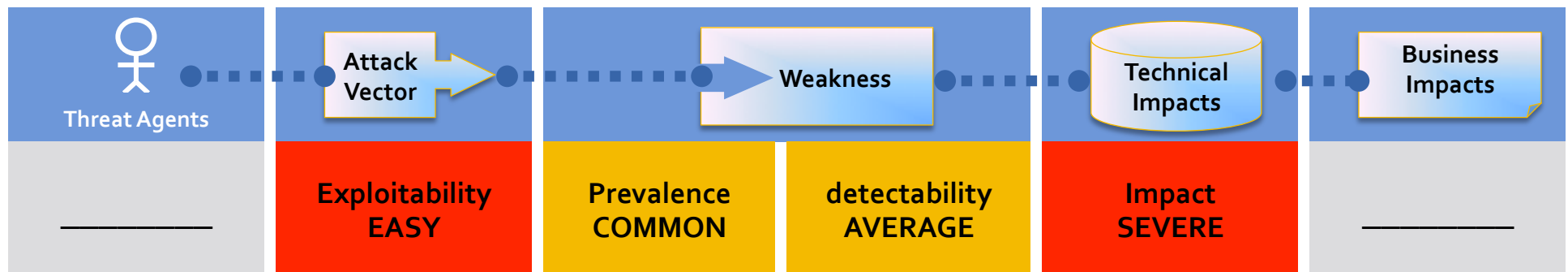
- OWASP provide a list of the top ten 'risks'.
- The names of the risks in the Top 10 stem from the type of attack, the type of weakness, or the type of impact they cause. OWASP chose the name that is best known and will achieve the highest level of awareness.
- The primary aim of the OWASP Top 10 is to educate.

How did OWASP Rank Risks?

- The basic standard risk model is
Risk = Likelihood * Impact
- The OWASP approach used in this document is based on standard methodologies and is customized for application security.

A1 –Injection

- Injection flaws, such as SQL, OS, and LDAP injection, occur when un-trusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing unauthorized data.



DEMO – CSS/HTML injection 1



<http://iona.net.nz/leaveMsg.asp>

<http://iona.net.nz/leaveMsgPost.asp>

LeaveMsg.asp

- Allows user to enter a message for the bank
- Only one message per user at a time



confirmMsg.asp

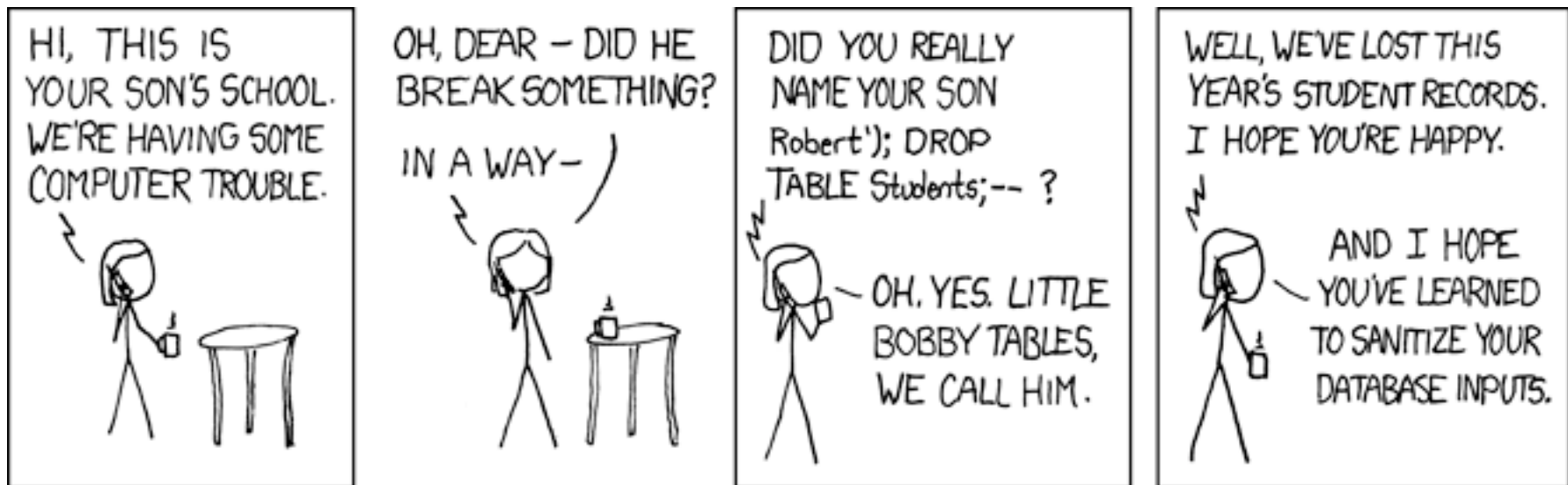
- Saves message into queue (form's target)
- Confirms message saved



Email.asp

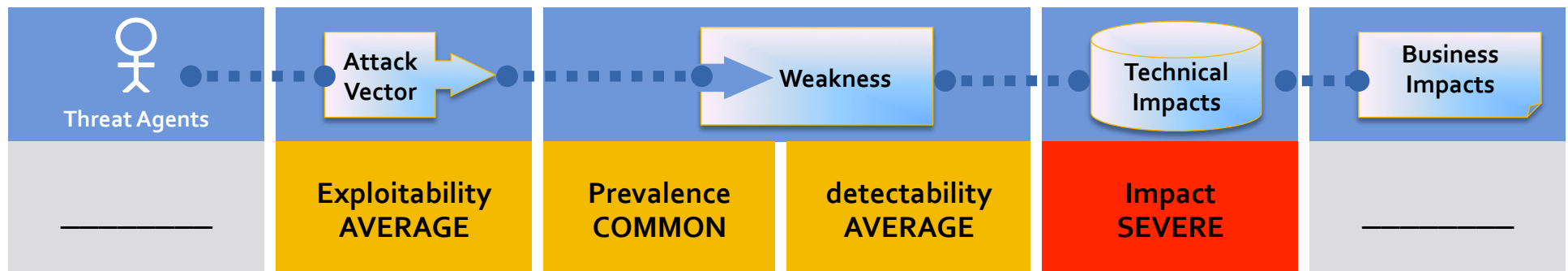
- Generates HTML for email to user

SQL Injection

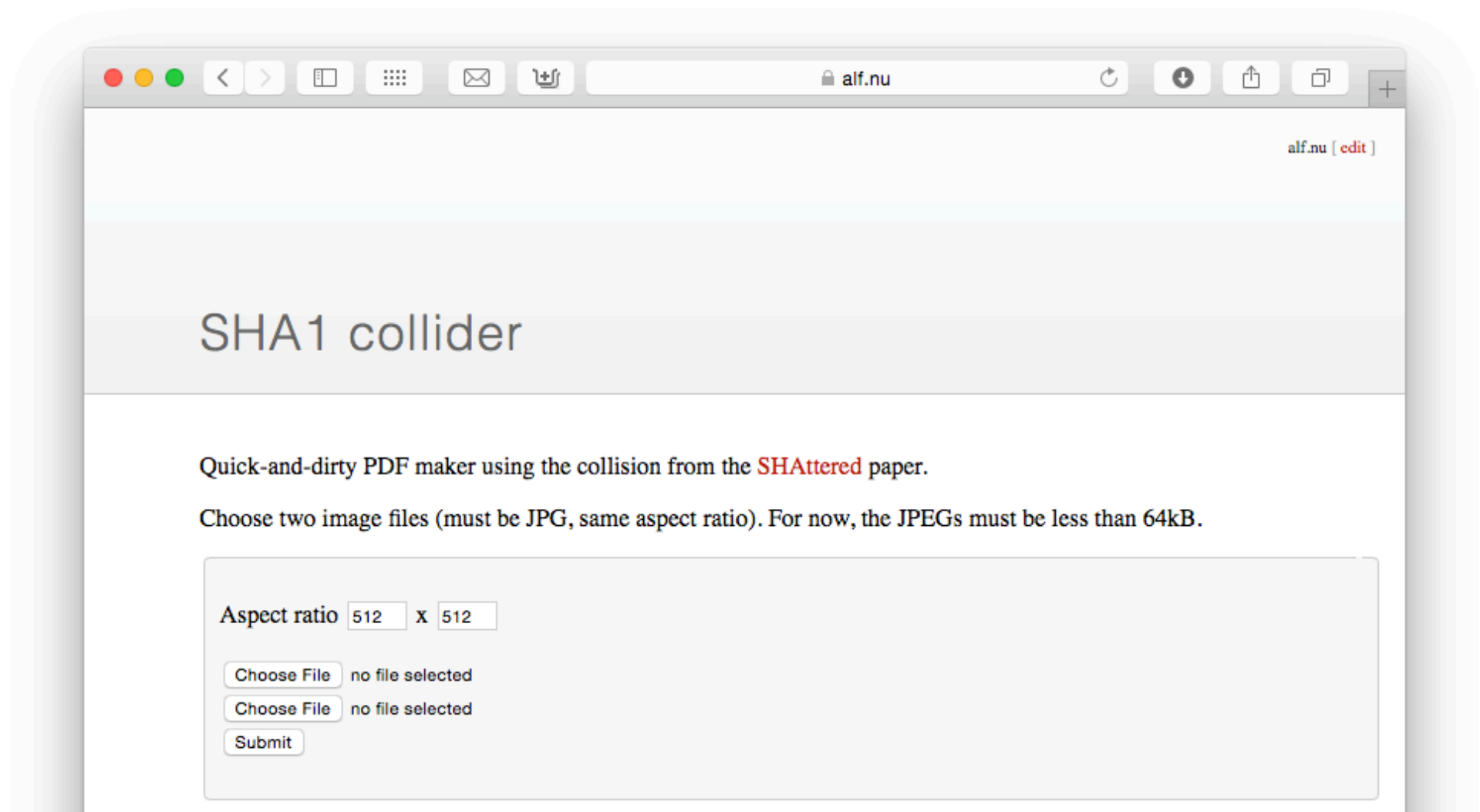


A2 –Broken Authentication and Session Management

- Application functions related to authentication and session management are often not implemented correctly, allowing attackers to compromise passwords, keys, session tokens, or exploit other implementation flaws to assume other users' identities.

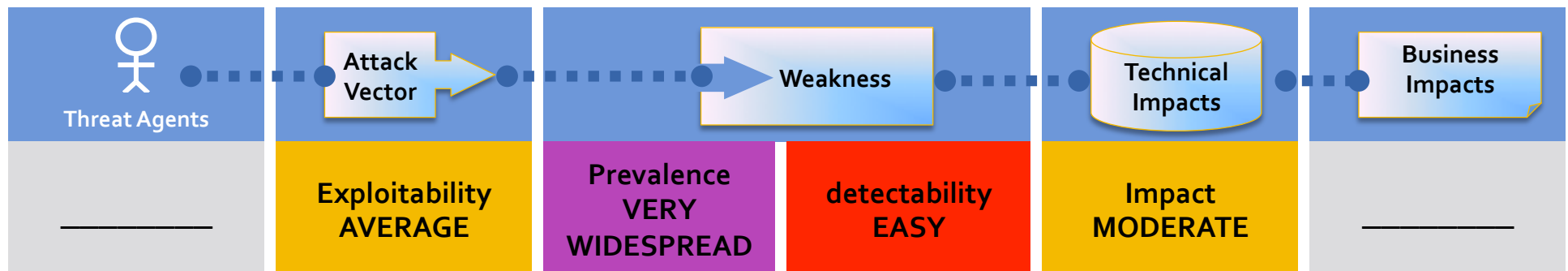


New threats - Hash Collision



A₃ –Cross-Site Scripting (XSS)

- XSS flaws occur whenever an application takes un-trusted data and sends it to a web browser without proper validation and escaping. XSS allows attackers to execute scripts in the victim's browser which can hijack user sessions, deface web sites, or redirect the user to malicious sites.



DEMO – XSS



<http://iona.net.nz/leaveMsgPost.asp>



Example – XSS via Wi-Fi SSID

Vulnerability Notes Database

Advisory and mitigation information about software vulnerabilities

[DATABASE HOME](#)[SEARCH](#)[REPORT A VULNERABILITY](#)[HELP](#)

Vulnerability Note VU#101462

DrayTek Vigor 2700 ADSL router contains a command injection vulnerability

Original Release date: 22 Oct 2013 | Last revised: 22 Oct 2013

[Print](#)[Tweet](#)[Send](#)[Share](#)

Overview

DrayTek Vigor 2700 ADSL router version 2.8.3 and possibly earlier versions contain a command injection vulnerability via malicious SSID ([CWE-77](#)).

Description

CWE-77: Improper Neutralization of Special Elements used in a Command ("Command Injection")

DrayTek Vigor 2700 ADSL router version 2.8.3 contains a command injection vulnerability. The Vigor router stores discovered SSIDs of surrounding access points into the `swlessSurvey` variable inside `variables.js`, which is a list of multiple variables used by the web administration console. An attacker can utilize JavaScript code in the SSID value of a malicious nearby access point which will cause the DrayTek router to execute the command when it adds the SSID into the `variables.js` list.

Impact

An attacker within range of the DrayTek Vigor ADSL router can edit the SSID on their malicious access point to corrupt the `variables.js` file. This may cause the DrayTek router to call external scripts or make unauthorized changes to

Quick Search

[Advanced Search »](#)

View Notes By

- [Date Published](#)
- [Date Public](#)
- [Date Updated](#)
- [CVSS Score](#)

Report a Vulnerability



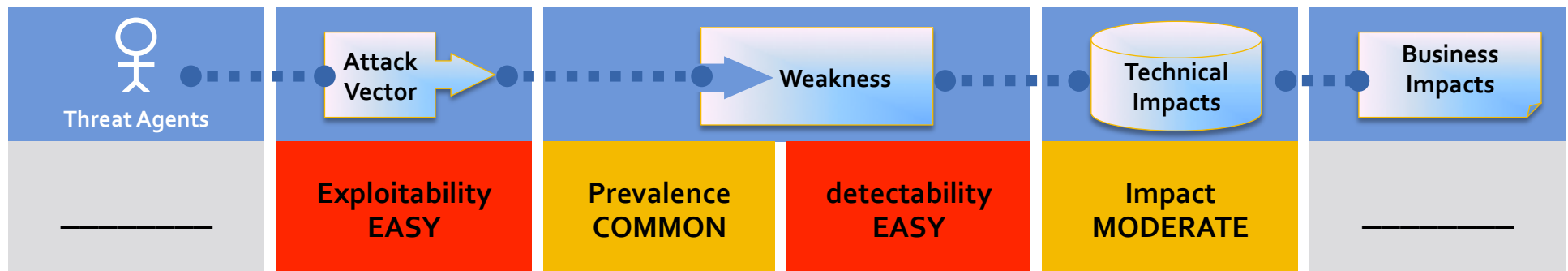
Please use the [Vulnerability Reporting Form](#) to report a

vulnerability. Alternatively, you can send us email. Be sure to read our [vulnerability disclosure policy](#).

Connect with Us

A4 –Insecure Direct Object References

- A direct object reference occurs when a developer exposes a reference to an internal implementation object, such as a file, directory, or database key. Without an access control check or other protection, attackers can manipulate these references to access unauthorized data.



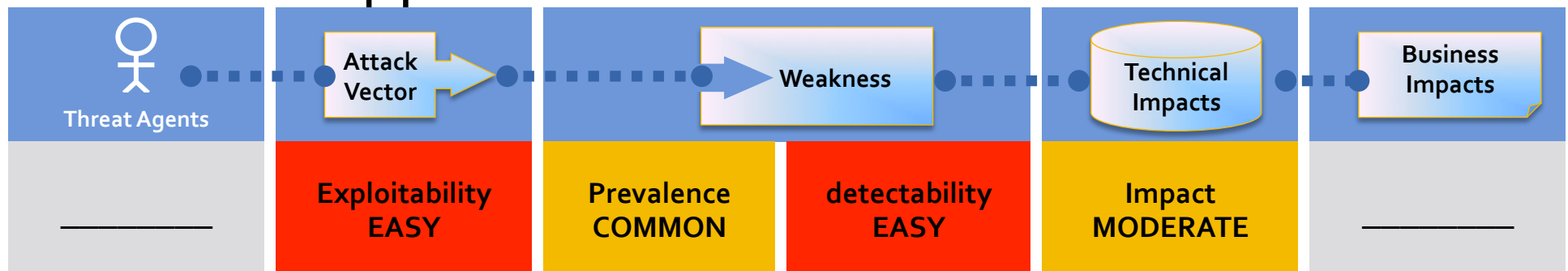
DEMO –Insecure Direct Object References Examples



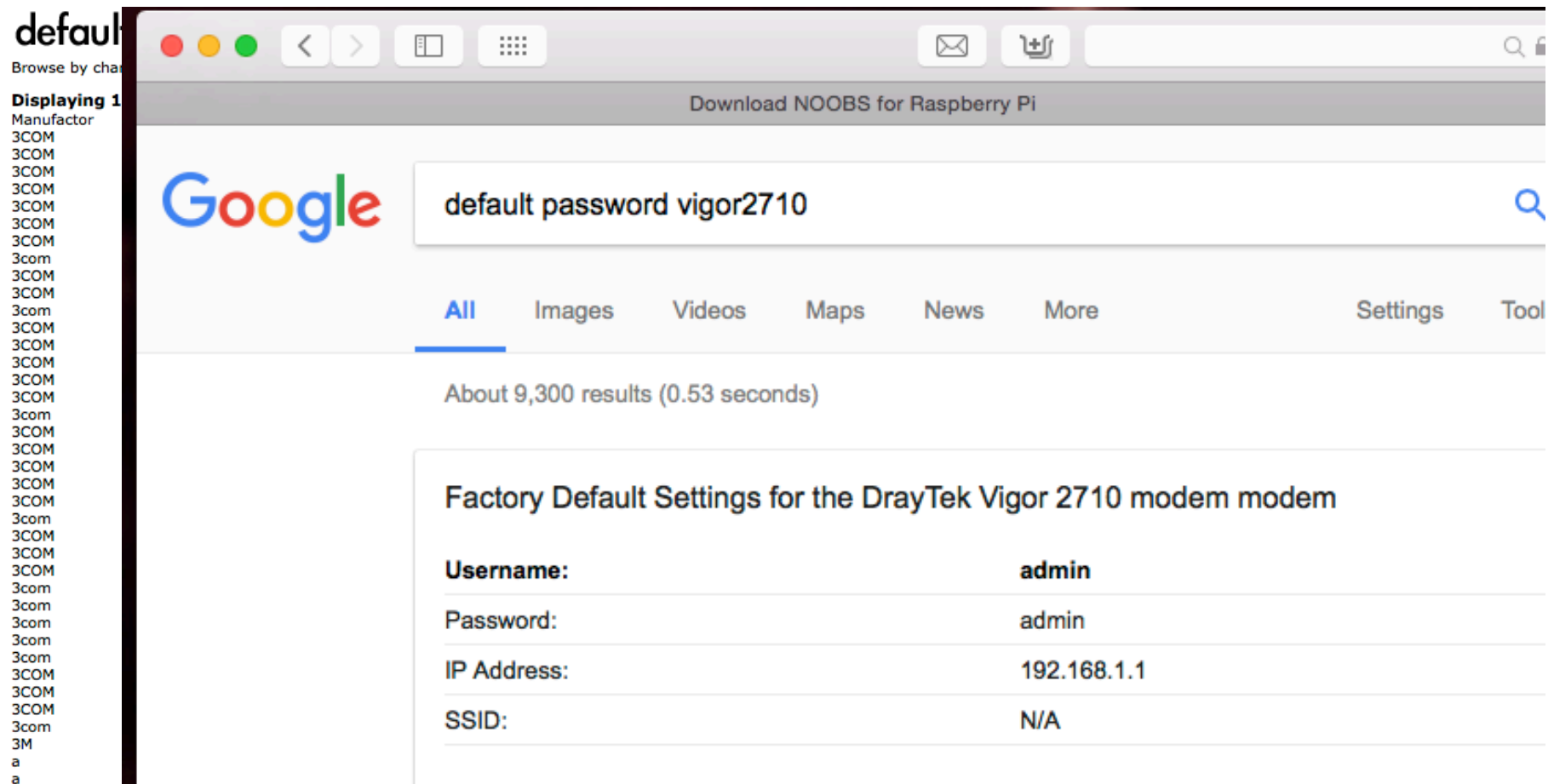
- Example Ecan bus website
 - <http://www.stuff.co.nz/the-press/news/transport/9386649/ECan-public-transport-card-hacked>
 - Demo of how you might have hacked the site using the command line and Excel
 - My card number is 733731 and I can see my balance at <http://iona.net.nz/bus.asp?id=733731>

A5 –Security Misconfiguration

- Good security requires having a secure configuration defined and deployed for the application, frameworks, application server, web server, database server, and platform. All these settings should be defined, implemented, and maintained as many are not shipped with secure defaults.

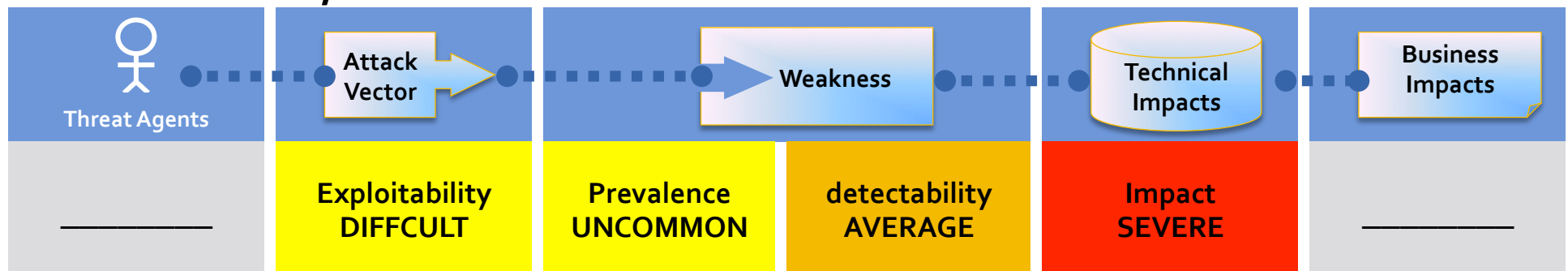


- <http://www.defaultpassword.com>

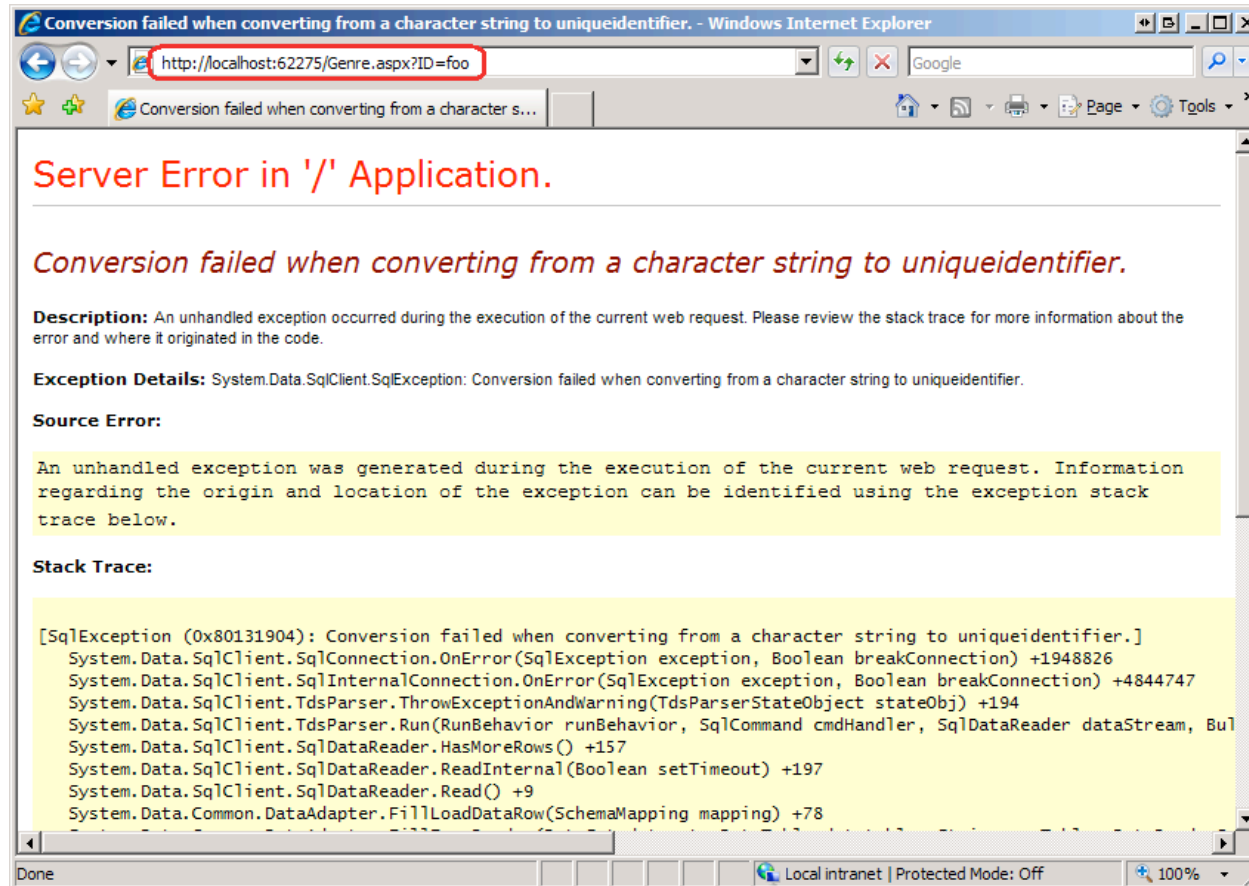


A6 – Sensitive Data Exposure

- Many web applications do not properly protect sensitive data, such as credit cards, SSNs, and authentication credentials, with appropriate encryption or hashing. Attackers may steal or modify such weakly protected data to conduct identity theft, credit card fraud, or other crimes.



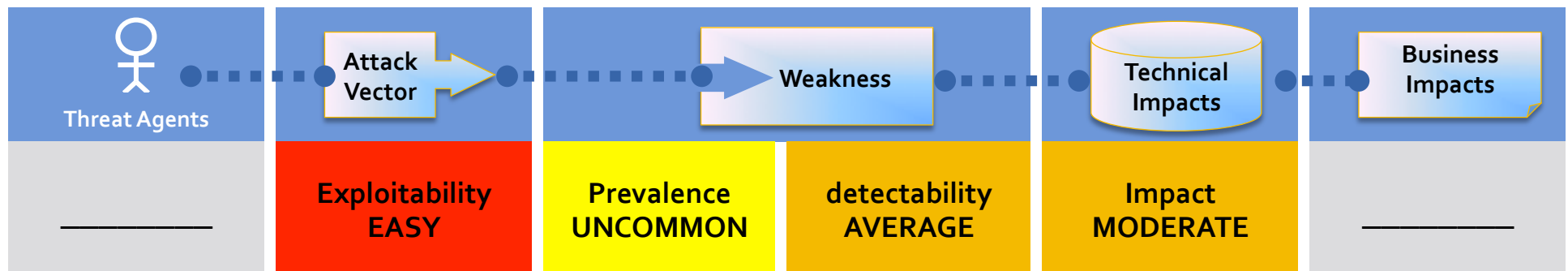
A6 –Sensitive Data Exposure Examples



```
?013-11-04 12:01:15,718 [6012] DEBUG Thud.ThudQueueMonitor [(null)] - Valid login by [redacted]
?013-11-04 12:01:25,828 [6012] DEBUG Thud.ThudQueueMonitor [(null)] - Processing request
?013-11-04 12:01:25,828 [6012] DEBUG Thud.ThudQueueMonitor [(null)] - <loginRequest version="1"><sessionId>917305940</sessionId><delivery method="web"><format>Full XML</format><address>h
?013-11-04 12:01:25,828 [6012] DEBUG Thud.ThudQueueMonitor [(null)] - Executing spLoginUser ' [redacted] ', '917305940'
?013-11-04 12:01:25,890 [6012] DEBUG Thud.ThudQueueMonitor [(null)] - Executing spGetPermissions be11386d-c65a-4b6f-8c4c-6c65e75a68b5
?013-11-04 12:01:25,890 [6012] DEBUG Thud.ThudQueueMonitor [(null)] - Executing spGetUserPermissions be11386d-c65a-4b6f-8c4c-6c65e75a68b5, -1
```

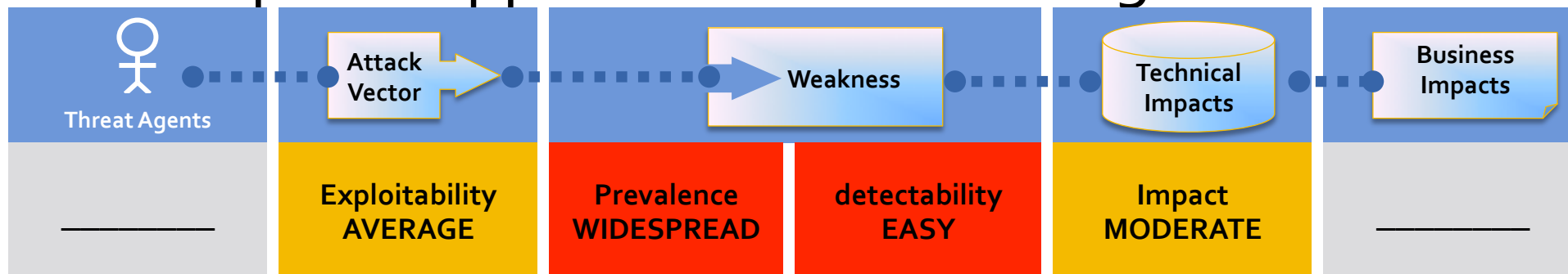
A7 –Missing Function Level Access Control

- Not all users should have access to all functions. Sometimes, function level protection is managed via configuration, and the system is misconfigured or sometimes, developers forget to include the proper checks.



A8 –Cross-Site Request Forgery (CSRF)

- A CSRF attack forces a logged-on victim's browser to send a forged HTTP request, including the victim's session cookie and any other automatically included authentication information, to a vulnerable web application. This forces the victim's browser to generate requests application thinks are legitimate.



DEMO – CSRF



SetupRouter.com

Username

Password

Login

Copyright© DrayTek Corp. All Rights Reserved. **DrayTek**

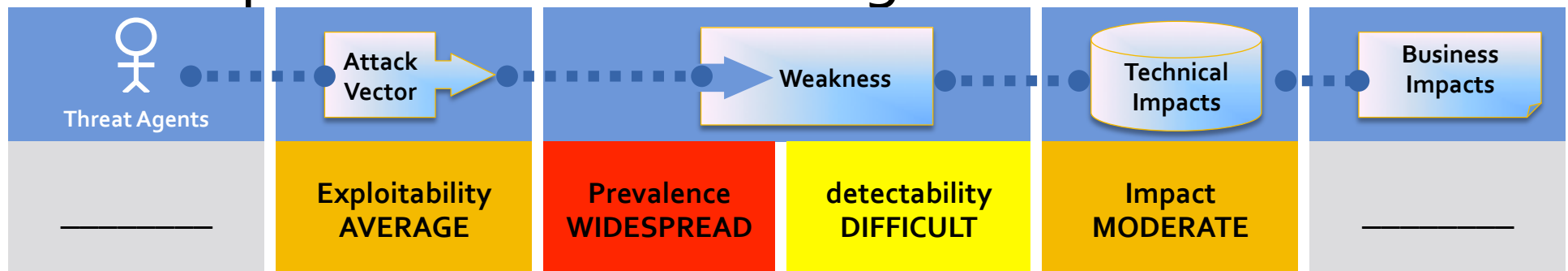


DEMO – CSRF



Ag –Using Components with Known Vulnerabilities

- Virtually every application has these issues because most development teams don't focus on ensuring their components/libraries are up to date. In many cases, the developers don't even know all the components they are using, never mind their versions. Component dependencies make things even worse.

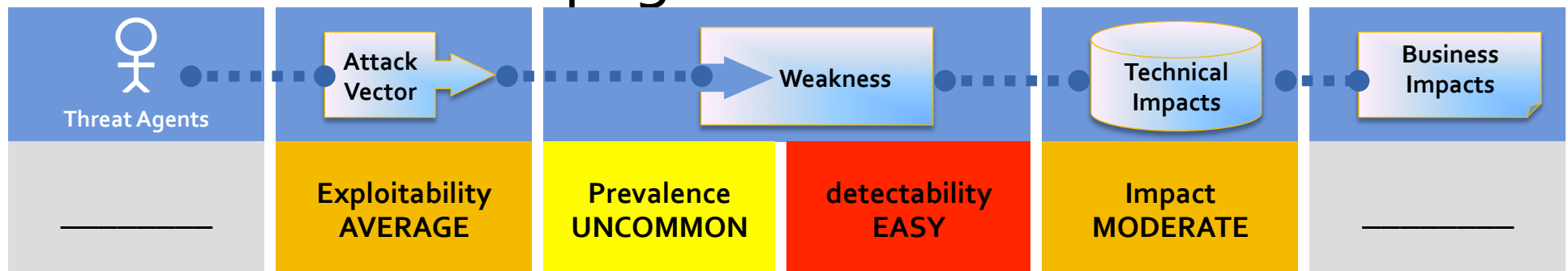


Example:



A10 –Unvalidated Redirects and Forwards

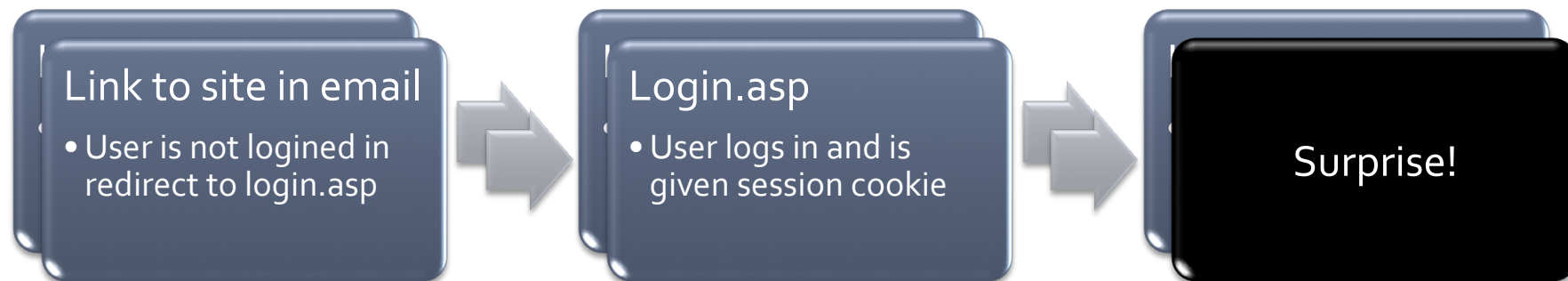
- Web applications frequently redirect and forward users to other pages and websites, and use un-trusted data to determine the destination pages. Without proper validation, attackers can redirect victims to phishing or malware sites, or use forwards to access unauthorized pages.



DEMO – Unvalidated Redirects



<http://iona.net.nz/home.asp>



Conclusion

- The OWASP Top 10 is a place to start. It does **not** define all the possible threats.
- The Top 10's roll is to **educate** and provide an **introduction** a number of critical threats.
- Testing for weaknesses does not provide as much assurance as having a secure development lifecycle.
- Build security in don't "bolt it on".

Tools used in demos



Firebug

- <https://getfirebug.com>



HttpFox

- <https://addons.mozilla.org/en-US/firefox/addon/httpfox>



Tamper Data

- <https://addons.mozilla.org/en-US/firefox/addon/tamper-data>



XSS Me

- <https://addons.mozilla.org/en-US/firefox/addon/xss-me>



HackBar

- <https://addons.mozilla.org/en-US/firefox/addon/hackbar>



Piggy bank's pages

- Login.asp – Access for authorised users
 - admin admin or user password
 - Home.asp – Home of piggy bank's Intranet
 - LeaveMsg.asp – Leave a message (GET)
 - LeaveMsgPost.asp - Leave a message (POST)
 - ConfirmMsg.asp – confirms message
 - Email.asp – generates HTML for email
 - DisplayMsg.asp – Banks view of waiting messages
-
- Base URL <http://iona.net.nz/>